Shopping Droid – Node-RED Data Base Documentation

1. Introduction

Node-RED is used as the flow-based programming environment for managing communication, processing robot data and interacting with the Shopping Droid database.

It connects the robot's MQTT messages to the database and provides flows for handling telemetry, offline data and shopping management.

2. Technologies Used

Technology	Purpose
Node-RED	Flow-based system integration
MQTT	Communication between the ESP32 and Node-RED
Mosquitto	MQTT message broker
SQLite	Local database storage
HTTP	Testing data synchronisation
JSON	Message data format

3. System Architecture

The system follows this communication structure:

ESP32 → Wi-Fi → Mosquitto MQTT Broker → Node-RED → SQLite Database

The ESP32 publishes robot information through MQTT. Node-RED receives the messages, processes them and stores the information in SQLite.

4. MQTT Communication

The MQTT broker used during development is Mosquitto.

Configuration:

Setting	Value
Broker address	127.0.0.1
Port	1883
MQTT topic	droid/sensor

The topic is used to receive sensor information from the Shopping Droid.

Example test message:

```
{
  "distance": 25,
  "unit": "cm"
}
```

This is a sample message used for testing, not a confirmed live sensor reading.

5. Node-RED Flows

5.1 Sensor Data Flow

This flow receives MQTT messages from the robot.

The process is:

1. MQTT input subscribes to droid/sensor.
2. JSON processing converts the incoming message into usable data.
3. A Function node processes the information.
4. SQLite stores the telemetry data.

5.2 Offline Queue Flow

This flow supports temporary storage when data cannot immediately be synchronised.

The process is:

1. Incoming data is received.
2. Data is inserted into the offline_queue table.
3. Pending messages remain stored locally.
4. The retry flow can process pending records for synchronisation.

5.3 Retry and Synchronisation Flow

This flow is designed to send queued messages to an HTTP endpoint.

The process is:

1. Retrieve pending messages from SQLite.
2. Send messages using an HTTP POST request.
3. Use the local test endpoint:

http://localhost:1880/shopping-droid/sync-test

4. Update the database records with retry and synchronisation information.

The complete retry and synchronisation process still requires testing.

5.4 HTTP Test Receiver

An HTTP endpoint is included for testing incoming synchronisation requests.

It allows the system to test how messages are received through HTTP.

5.5 Shopping Droid Management Flow

This flow contains management functionality for the shopping system, including:

- Supplier inventory.
- Customer orders.
- Inventory updates.
- Order items.

These functions support the shopping and delivery management side of the project.

6. Database Integration

Node-RED uses SQLite for local data storage.

The database contains the following main tables:

Table	Purpose
telemetry	Stores robot sensor information
offline_queue	Stores messages awaiting synchronisation
inventory	Stores product inventory
orders	Stores customer orders
order_items	Stores products belonging to orders

The database files are maintained in the existing project:

- database.db
- database_schema.sql

7. Running the System

Step 1: Start Mosquitto

In Ubuntu:

sudo service mosquitto start

Step 2: Start Node-RED

node-red

Step 3: Open Node-RED

Open the Node-RED editor in a browser:

http://localhost:1880

Step 4: Deploy

Open the existing flows and select **Deploy** to apply changes.

Step 5: Test MQTT

Use the following command in another Ubuntu terminal:

mosquitto_pub -h 127.0.0.1 -t droid/sensor -m '{"distance":25,"unit":"cm"}'

This sends a test message to the MQTT broker.

Check the relevant Node-RED flow and database to verify whether the message is received and stored.

8. Project Integration

Node-RED forms the communication and data-management layer of the Shopping Droid.

It connects with the other project components:

- **XOD:** Develops the ESP32 firmware.
- **ESP32:** Collects and transmits robot information.
- **MQTT:** Transfers messages.
- **Mosquitto:** Handles MQTT communication.
- **Node-RED:** Processes incoming messages and manages flows.
- **SQLite:** Stores telemetry, orders and inventory information.

9. Current Implementation Status

Component	Status
Node-RED environment	Set up
MQTT broker configuration	Configured

Sensor data flow	Created
JSON processing	Included
SQLite integration	Configured
Offline queue flow	Created
Retry and synchronisation flow	Created
HTTP test receiver	Created
Shopping management flow	Created

Conclusion

Node-RED provides the integration layer for the Shopping Droid system. It receives MQTT messages, processes robot data, stores information in SQLite and supports offline message handling and synchronisation.

The existing flows and database provide the foundation for integrating the XOD firmware and physical Shopping Droid hardware.

Shopping Droid – XOD Firmware Documentation

1. Introduction

The Shopping Droid project uses an ESP32 microcontroller to support communication between the robot and the central management system.

XOD is used as a visual programming environment to develop the ESP32 firmware. The firmware is designed to connect the ESP32 to a Wi-Fi network and publish robot sensor information through MQTT.

2. Development Environment

The following software was used:

- **XOD IDE:** Visual programming environment used to develop the firmware.
- **ESP32:** Microcontroller selected for the robot.
- **MQTT:** Messaging protocol used to transmit data.
- **Node-RED:** Receives and processes MQTT messages.

- **Mosquitto:** MQTT broker.
- **SQLite:** Stores robot data in the existing database.

3. XOD Project Setup

A new XOD project was created with the name:

shopping-droid-firmware

The main patch was used to organise the firmware nodes.

The project was saved and exported as an XOD bundle:

shopping driod XOD Database-and-Offline-Synchronisation.xodball

The exported file was copied from the Windows Downloads folder into the existing GitHub project under:

Shopping-Droid-MQTT-Database-and-Offline-Synchronisation/firmware/

4. ESP32 Wi-Fi Configuration

The esp32/connect node was added to the main XOD patch.

Two constant-string nodes were connected to its inputs:

- SSID: Specifies the Wi-Fi network name.
- PWD: Specifies the Wi-Fi password.

A Boolean constant was also used to enable the connection.

The actual Wi-Fi credentials are entered locally and are not included in this documentation.

5. MQTT Configuration

The MQTT functionality was added using the following XOD nodes:

- nusabot/device
- nusabot/connect
- mqtt-publish

The nusabot/device node provides the MQTT device configuration.

The following settings were configured:

Setting	Configuration
Server	192.168.18.40

Device ID	shopping_droid_01
Username	Left empty
Password	Left empty

The nusabot/connect node is used to establish the MQTT connection.

The MQTT publish node was configured with:

Input	**Value**
Topic	droid/sensor
Payload	{"distance":25,"unit":"cm"}
QoS	0
Retain	false

The payload is currently a test value. It will need to be replaced with actual sensor readings when the sensor integration is completed.

6. MQTT Communication

The firmware is intended to publish messages using the topic:

droid/sensor

The communication architecture is:

ESP32 → Wi-Fi → MQTT Broker → Node-RED → SQLite Database

Node-RED receives the published messages and processes them using the existing MQTT and database flows.

7. Ultrasonic Sensor

The HC-SR04 ultrasonic sensor was considered for measuring the distance between the robot and nearby obstacles.

An XOD patch named hc-sr04 was created.

However, the sensor implementation was left for another team member to complete. The current firmware therefore uses a test payload rather than live sensor measurements.

8. Current Implementation Status

Component	**Status**

XOD project created	Completed
Main patch created	Completed
ESP32 Wi-Fi node added	Completed
Wi-Fi input constants connected	Completed
MQTT device configuration added	Completed
MQTT publish node added	Completed
MQTT topic configured	Completed
Test payload configured	Completed
QoS and retain configured	Completed

9. GitHub Project Structure

The firmware is stored in the existing project without changing its other files.

```
egrocca/
└── Shopping-Droid-MQTT-Database-and-Offline-Synchronisation/
    ├── database.db
    ├── database_schema.sql
    ├── flows.json
    ├── README.md
    ├── screenshots/
    └── firmware/
        └── shopping driod XOD Database-and-Offline-Synchronisation.xodball
```

Conclusion

The XOD firmware project has been created and exported. The ESP32 Wi-Fi and MQTT publishing components have been configured using visual programming nodes.